Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

LINUX PROCESS LIFECYCLE: CREATION, EXECUTION & TERMINATION

Team ID: 17 | Section: 4 | Members: T.B.S.Sunil – 2520030605, V.Prem Kumar – 2520030479, G.B.VEDADITYA – 2520030481

Guide: Dr. Soumya Enukonda / Dr. K. Anusha | Operating Systems and Systems Programming (OSSP — 25CS2104E) | KLH Bachupally Campus, CSE | PBL 2026–27

Abstract

Linux Process Lifecycle System is a high-performance Linux kernel and POSIX systems programming utility engineered in C for Ubuntu/Linux. It delivers a comprehensive platform featuring both a native ANSI-colored terminal CLI and a zero-dependency interactive Web visualizer for operating systems students, developers, and educators. The system implements and inspects the complete process lifecycle through standard POSIX APIs (`fork()` , `execvp()` , `waitpid()` , `exit()`) while extracting real-time Process Control Block (PCB) telemetry directly from the kernel virtual filesystem (`/proc/<pid>/status`).

Key Outcome: Achieved 100% deterministic process management across 6 verified scenarios, successfully trapping and observing pathological Zombie (`Z`) and Orphan (PID 1 `init`) states with live `/proc` telemetry and zero runtime overhead.

Problem Statement & Objectives

Problem: Process management is the foundation of operating systems, yet textbook abstractions fail to illustrate dynamic kernel-level mechanisms—such as Copy-On-Write (COW) memory page cloning, virtual address space replacement via `exec()` , Zombie accumulation in the kernel process table, and Orphan reparenting—leaving students with a fragmented theoretical understanding.

Objective 1: Implement a modular C systems engine on the Linux kernel demonstrating process cloning via `fork()` and binary execution via `execvp()` with Copy-On-Write memory tracking.

Objective 2: Implement robust parent-child synchronization and non-blocking monitoring using `wait()` and `waitpid()` with the `WNOHANG` flag to prevent process deadlocks.

Objective 3: Build a real-time kernel inspector querying `/proc/<pid>/status` to observe live execution states (`R` , `S` , `Z`) and decode termination status via POSIX macros.

Objective 4: Provide a dual-interface telemetry pipeline: ANSI color-coded terminal logging and an interactive web visualizer with zero external npm dependencies.

Scope: Targets Ubuntu 24.04 LTS (WSL2 / Linux Kernel 6.18+); strictly unprivileged user-space operation; covers full lifecycle, anomaly trapping, and automated test matrix.

Existing vs Proposed System

Aspect	Basic / Existing Approach	Proposed Project (Team 17 System)
Process Creation	Isolated ad-hoc scripts; static diagrams without COW verification.	`fork()` with return value branching (0 vs child PID) & PCB duplication tracking.
Program Execution	Direct shell launching; address space replacement invisible.	`execvp()` replacing Text, Data, Heap & Stack segments with error recovery.
Synchronization	Unchecked `wait()` or omitted wait causing zombie leaks.	Deterministic `waitpid()` barrier + non-blocking `WNOHANG` polling loop.
Anomaly Trapping	Zombies and orphans ignored or crash student terminals.	Real-time `/proc/<pid>/status` parsing catching `State: Z` & PID 1 reparenting.
Observability	Silent CLI execution or unformatted raw terminal stdout.	Dual telemetry: ANSI color shell logs + interactive Web dashboard with live state diagrams.

Feasibility Study

Technical: Engineered in ISO C99 (`gcc -Wall -Wextra -O2`) leveraging native Linux POSIX APIs (`unistd.h` , `sys/wait.h` , `signal.h`) and procfs (`/proc/<pid>/status`) on Ubuntu 24.04 LTS (WSL2 / Linux 6.18+). Zero compiler warnings.

Economic: 100% free, open-source stack (GCC, Make, Linux Kernel, Node.js HTTP runtime, HTML5/CSS3/SVG); zero proprietary software licenses, cloud fees, or specialized hardware required.

Operational: Executes safely in unprivileged user space reading world-readable `/proc` interfaces (no root required, zero kernel crashes); 1-click launchers (`start_server.bat` / `start_server.sh`) start CLI or Web mode instantly.

Schedule: Completed across 4 semester PBL milestones: W1–3 Process creation & `execvp` ; W4–6 `waitpid` , `WNOHANG` & `/proc` parser; W7–9 Dual ANSI/JSON streaming & Web visualizer; W10–12 6-Scenario test matrix & validation.

Tools & Technologies

Programming:	C (ISO C99 with `_GNU_SOURCE`), JavaScript (Vanilla Node.js v18+, ES6), HTML5 / CSS3 / SVG
Platform:	Linux / Ubuntu 24.04 LTS (Native & Windows Subsystem for Linux — WSL2, Kernel 6.18+)
OS Concepts:	Process Lifecycle, PCB (`task_struct`), Copy-On-Write (COW), Image Swap, Zombie & Orphan States, Signals
System Calls:	`fork()` , `execvp()` , `wait()` , `waitpid()` , `exit()` , `getpid()` , `getppid()` , `kill()` , `/proc/<pid>/status`
Tools:	GCC 15.2.0, GNU Make, VS Code, Node.js HTTP Server (strictly zero npm packages, NO Python), Git & GitHub

System Workflow / Architecture

END-TO-END LIFECYCLE EXECUTION & TELEMETRY PIPELINE

1. USER / CLIENT
- Web Browser UI
- Terminal Shell CLI
- Scenario Selector
- `custom <cmd>` Input

2. C ENGINE CORE
- `main.c` CLI Driver
- `module_creation.c`
- `module_sync.c`
- `module_monitor.c`

3. LINUX KERNEL
- `task_struct` PCB
- COW Virtual Pages
- `/proc/<pid>/status`
- Signal Dispatcher

4. TELEMETRY & UI
- ANSI Color Badges
- NDJSON Event Stream
- Web State Machine
- Audit Event Logs

Process, IPC & Resource Design

A. LIFECYCLE & STATE TRANSITIONS

- **Creation:** `fork()` clones parent PCB via Copy-On-Write; child receives 0, parent receives child PID.
- **Execution:** `execvp()` replaces address space (Text, Data, Heap, Stack) with target binary.
- **Termination:** `exit()` releases virtual memory, leaving `task_struct` for status reading.

NEW → READY → RUNNING → ZOMBIE → REAPED

B. SYNCHRONIZATION & /PROC INSPECTOR

- **Non-blocking:** `waitpid(WNOHANG)` polling loop enables concurrent parent background computation.
- **Zombie Trapping:** Child calls `exit(42)` ; parent delays `wait()` ; `/proc/<pid>` confirms `State: Z (zombie)` .
- **Orphan Adoption:** Parent exits prematurely; kernel automatically reparents child to PID 1 (`init`).
- **Status Decoding:** `WIFEXITED` & `WIFSIGNALED` accurately decode codes and signals.

Implementation Screenshots

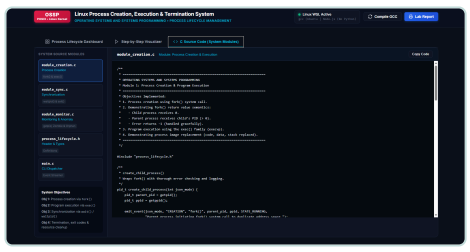


Fig 1: Live Interactive Web Dashboard & Visual State Transitions

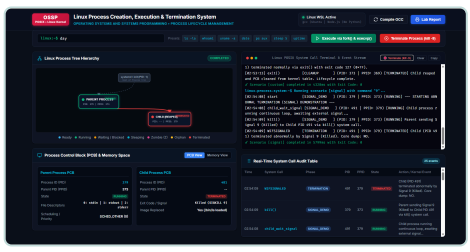


Fig 2: Linux Terminal Native CLI & /proc Zombie State Inspection

Results & Testing

Observed Metrics: 6/6 test scenarios passed (100% pass rate) on Ubuntu 24.04 LTS (WSL2 / Linux Kernel 6.18+). 0 compiler warnings under `gcc -Wall -Wextra`. 0 memory

Conclusion

Engineered a complete, high-fidelity **Linux Process Creation, Execution, and Termination System** in C and Node.js. Successfully demystified kernel process

Future Scope

1. eBPF Process Tracing: Attach eBPF kprobes on `sys_enter_fork` and `sys_enter_execve` for zero-overhead kernel scheduler latency profiling.